

INDIAN INSTITUTE OF TECHNOLOGY HYDERABAD

M.Tech Computer Science and Engineering

**CS6890 · FRAUD ANALYTICS USING PREDICTIVE AND SOCIAL NETWORK
TECHNIQUES**

Fraud Detection using Generative Adversarial Networks

Assignment 4

Akarsh Dubey CS25MTECH14001

Atish Kadam CS25MTECH14003

Code Link:

<https://colab.research.google.com/drive/1MrnbJLBgOVNvPrz80aV5bDZHLqCuj8XJ?usp=sharing>

April 30, 2026

Contents

1	Introduction	2
2	Mathematical Background	2
2.1	Generative Adversarial Networks (GANs)	2
2.2	BiGAN — Bidirectional GAN	2
2.3	AnoGAN — Anomaly GAN	3
2.4	Threshold Selection and Evaluation	3
3	Dataset Description	4
3.1	Data Splitting Strategy	4
3.2	Preprocessing Pipeline	4
4	Model Architecture and Hyperparameters	5
4.1	BiGAN Architecture	5
4.2	AnoGAN Architecture	5
4.3	Hyperparameters	6
5	Implementation	6
5.1	Data Splitting and Preprocessing	6
5.2	Training Setup	7
5.3	BiGAN and AnoGAN Scoring	7
5.4	Threshold Selection and Evaluation	7
6	Experimental Setup	8
7	Results and Discussion	8
7.1	Training Convergence	8
7.2	Anomaly Score Distributions	9
7.3	Per-Model Diagnostic Plots	9
7.4	Quantitative Results on Test Set	10
7.5	BiGAN vs. AnoGAN Comparison	10
7.6	Discussion	11
8	Algorithmic Summary	12
9	Conclusion	13

1. Introduction

Credit card fraud detection is a critical challenge in financial security. The rarity of fraudulent transactions (typically less than 0.2% of all transactions) makes this an inherently imbalanced classification problem. Traditional supervised approaches require labelled fraud examples, which may be scarce or unavailable in real-world deployments.

This assignment adopts an **unsupervised anomaly detection** paradigm using Generative Adversarial Networks (GANs). Both models are trained exclusively on *normal* transactions and learn the underlying data manifold; at inference time, transactions that cannot be well-reconstructed or explained by the learned distribution receive high anomaly scores and are classified as fraud.

- **Problem:** Detect fraudulent credit card transactions from a highly imbalanced dataset (492 frauds out of 284,807 total transactions, $\approx 0.173\%$) without using fraud labels during training.
- **Goal:** Implement and evaluate two GAN-based anomaly detectors — BiGAN (Bidirectional GAN) and AnoGAN — on the Kaggle Credit Card Fraud dataset; compare them across Accuracy, Precision, Recall, F1-score, ROC-AUC, and AUPRC.
- **Approach:** Apply leak-free two-stage scaling, train each GAN on normal-only data, derive per-sample anomaly scores, select the F1-optimal classification threshold on a held-out validation set, and evaluate once on a separate test set.

2. Mathematical Background

2.1 Generative Adversarial Networks (GANs)

A standard GAN trains a Generator G and a Discriminator D in a minimax game:

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

where p_{data} is the real data distribution and $p_z = \mathcal{N}(0, I)$ is the latent prior. G maps latent vectors $z \in \mathbb{R}^{d_z}$ to data space $\hat{x} \in \mathbb{R}^{d_x}$, while D tries to distinguish real from generated samples. In anomaly detection, only normal data is used during training; anomalies lie off the learned manifold and thus yield high reconstruction error.

2.2 BiGAN — Bidirectional GAN

BiGAN augments the standard GAN with an **Encoder** $E : \mathbb{R}^{d_x} \rightarrow \mathbb{R}^{d_z}$ that maps real data back into latent space. The discriminator operates on *joint* pairs (x, z) :

$$\min_{G, E} \max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x, E(x))] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z), z))]$$

At convergence, $E \approx G^{-1}$, so a real sample x can be encoded and reconstructed as $\hat{x} =$

$G(E(x))$. The anomaly score combines reconstruction error, latent consistency, and a feature-space discriminator term:

$$A_{\text{BiGAN}}(x) = (1-\lambda) \left[\|x - G(E(x))\|_1 + w_{\text{lat}} \|E(x) - E(G(E(x)))\|_1 \right] + \lambda \|f_D(x, E(x)) - f_D(G(E(x)), E(x))\|_1$$

where $f_D(\cdot)$ denotes the 64-dimensional joint penultimate features of D , $\lambda = 0.1$, and $w_{\text{lat}} = 0.5$.

2.3 AnoGAN — Anomaly GAN

AnoGAN trains a standard Generator–Discriminator pair on normal data. At test time, for each query x^* , a latent vector z is optimised iteratively to minimise:

$$\mathcal{L}(z) = (1-\lambda) \|x^* - G(z)\|_2^2 + \lambda \|f_D(x^*) - f_D(G(z))\|_2^2$$

where $f_D(\cdot)$ is the 64-dimensional feature layer of the discriminator. The anomaly score is the minimum loss value after 500 Adam optimisation steps ($\text{lr}_z = 0.01$, $\lambda = 0.1$).

Remark

AnoGAN requires running a separate optimisation loop per test sample, making inference much slower than BiGAN’s single forward pass through the encoder. This is the fundamental speed–accuracy trade-off between the two approaches.

2.4 Threshold Selection and Evaluation

Because the dataset is heavily imbalanced, accuracy alone is insufficient. The classification threshold τ is chosen to maximise the F1-score on the validation set using the precision–recall curve:

$$\tau^* = \arg \max_{\tau} F_1(\tau) = \arg \max_{\tau} \frac{2 \cdot P(\tau) \cdot R(\tau)}{P(\tau) + R(\tau)}$$

τ^* is then applied *once* to the held-out test set, preventing any threshold peeking. Final evaluation uses Accuracy, Precision, Recall, F1-score, ROC-AUC, and AUPRC.

3. Dataset Description

The **Kaggle Credit Card Fraud Detection** dataset contains transactions made by European cardholders in September 2013 over two days.

Property	Description	Value
Rows / samples	Total credit card transactions	284,807
Features	Input features (30 used after dropping Class)	30
Normal class (0)	Legitimate transactions	284,315
Fraud class (1)	Fraudulent transactions	492
Fraud ratio	Fraction of fraudulent samples	0.1727%
Missing values	None	0

Input features: **Time** (seconds elapsed since the first transaction, raw scale 0–172,792), **V1–V28** (28 anonymised PCA-transformed features, already centred), **Amount** (transaction amount in euros, highly skewed), and **Class** (binary label used only for evaluation, never for training).

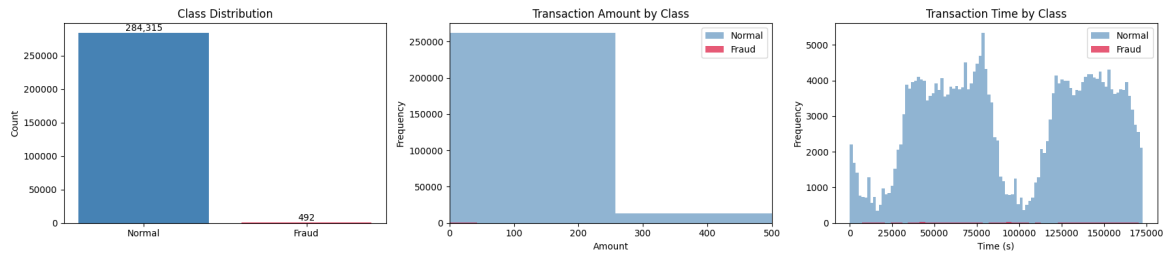


Figure 1: EDA: class distribution (left), transaction amount by class (centre), transaction time by class (right).

3.1 Data Splitting Strategy

Splitting is performed **before** any scaling to prevent data leakage. Scalers are fit exclusively on the training normal set.

Partition	Normal	Fraud
Train	213,236	0 (excluded)
Val	28,431	246
Test	42,648	246

3.2 Preprocessing Pipeline

1. **Stage 1** — **RobustScaler** on **Time** (col 0) and **Amount** (col 29), fit only on training normals. Handles the large scale difference between these raw features and the PCA features, preventing them from dominating the anomaly/reconstruction score.

2. **Stage 2 — StandardScaler** over all 30 features, fit on the Stage-1-transformed training normals. Ensures unit variance globally.
3. All validation and test splits use `.transform()` only. Final training normal mean: ≈ 0.0 , std: ≈ 1.0 .

4. Model Architecture and Hyperparameters

4.1 BiGAN Architecture

The BiGAN model contains three jointly trained networks: a Generator G , an Encoder E , and a Discriminator D . All networks use `LeakyReLU(0.2)` activations where applicable, with dropout used for regularisation in the Encoder and Discriminator.

Network	Layer Configuration	Params
Generator G	Linear(32→64) → BN → LReLU → Linear(64→128) → BN → LReLU → Linear(128→30)	14,686
Encoder E	Linear(30→128) → LReLU → Drop(0.3) → Linear(128→64) → LReLU → Drop(0.3) → Linear(64→32)	14,304
Discriminator D	x -branch: Linear(30→128), z -branch: Linear(32→64), Joint: Linear(192→64) → LReLU → Drop(0.3) → Linear(64→1)	18,497

The discriminator receives the joint pair (x, z) as input. Its penultimate 64-dimensional feature representation, denoted by $f_D(x, z)$, is used later for anomaly scoring.

4.2 AnoGAN Architecture

AnoGAN uses a standard Generator–Discriminator structure. Compared with the BiGAN Generator, the AnoGAN Generator is larger so that it can model the normal transaction manifold more expressively.

Network	Layer Configuration	Params
Generator G	Linear(32→64) → BN → ReLU → Linear(64→128) → BN → ReLU → Linear(128→256) → BN → ReLU → Linear(256→30) → Tanh	52,062
Discriminator D	Linear(30→256) → LReLU → Drop(0.3) → Linear(256→128) → LReLU → Drop(0.3) → Linear(128→64) → LReLU → Linear(64→1) → Sigmoid	49,153

4.3 Hyperparameters

Parameter	BiGAN	AnoGAN	Reason
Latent dimension d_z	32	32	Provides a compact but expressive latent representation.
Epochs	100	100	Allows sufficient convergence for tabular fraud-detection data.
Batch size	512	256	BiGAN uses a larger batch size because it learns joint (x, z) pairs.
LR Generator/Encoder	2×10^{-4}	2×10^{-4}	Adam optimiser with $(\beta_1, \beta_2) = (0.5, 0.999)$.
LR Discriminator	1×10^{-4}	1×10^{-4}	Uses TTUR-style slower discriminator updates for stability.
Reconstruction weight w_r	5.0	–	Encourages faithful reconstruction in the BiGAN anomaly score.
Feature-score weight λ	0.1	0.1	Controls the contribution of discriminator feature loss in anomaly scoring.
Latent consistency weight w_{lat}	0.5	–	Penalises mismatch between encoded and optimised latent representations.
Label smoothing	0.9	0.9	Reduces discriminator overconfidence during training.
Instance noise std	$0.1 \rightarrow 0$	$0.1 \rightarrow 0$	Noise is annealed from 0.1 to 0 over training epochs.
Gradient clipping	5.0	5.0	Prevents unstable updates due to gradient explosion.
z -optimisation iterations	–	500	AnoGAN inference optimises z using Adam with learning rate 0.01.

5. Implementation

The complete implementation was carried out in a single Jupyter notebook, `Team_45.ipynb`. The notebook includes data loading, preprocessing, model training, anomaly scoring, threshold selection, and final evaluation. Both BiGAN and AnoGAN were implemented as unsupervised anomaly detection models, where training was performed only on normal transactions.

The credit card fraud dataset contains 284,807 transactions, with 284,315 normal samples and only 492 fraud samples. Since fraud transactions form only about 0.1727% of the dataset, the problem is highly imbalanced. Therefore, metrics such as Precision, Recall, F1-score, ROC-AUC, and AUPRC were used along with Accuracy.

5.1 Data Splitting and Preprocessing

The dataset was split before scaling to avoid data leakage. Normal and fraud samples were separated first. Only normal samples were used for training, while validation and test sets contained both normal and fraud samples. The final split used 213,236 normal samples for training, 28,431 normal and 246 fraud samples for validation, and 42,648 normal and 246 fraud samples for testing.

A two-stage preprocessing method was used. First, `RobustScaler` was applied to the `Time` and `Amount` features because these features have different scales and may contain outliers. Then, `StandardScaler` was applied to all 30 input features. Both scalers were fitted only on the normal training data and then applied to validation and test data.

5.2 Training Setup

Both models used an input dimension of 30 and a latent dimension of 32. BiGAN was trained with batch size 512, while AnoGAN used batch size 256. Both models were trained for 100 epochs using the Adam optimiser with $(\beta_1, \beta_2) = (0.5, 0.999)$.

To improve GAN training stability, several techniques were used. Label smoothing was applied with real labels set to 0.9. Instance noise was annealed from 0.1 to 0 during training. A TTUR-style learning rate setup was used, with learning rate 2×10^{-4} for the generator side and 1×10^{-4} for the discriminator. Gradient clipping with maximum norm 5.0 was also applied.

5.3 BiGAN and AnoGAN Scoring

BiGAN used three networks: Generator G , Encoder E , and Discriminator D . During training, the discriminator learned to distinguish real joint pairs $(x, E(x))$ from generated pairs $(G(z), z)$. The Generator and Encoder were trained together using adversarial loss and reconstruction regularisation. After training, BiGAN anomaly scores were computed using reconstruction error, latent consistency error, and discriminator feature error.

AnoGAN used a standard Generator–Discriminator structure without an Encoder. During inference, a latent vector z was optimised for each input sample for 500 iterations using Adam with learning rate 0.01. The AnoGAN anomaly score combined reconstruction loss and discriminator feature loss. Because of this per-sample optimisation, AnoGAN inference was slower than BiGAN inference.

5.4 Threshold Selection and Evaluation

For both models, anomaly scores were first computed on the validation set. The precision–recall curve was used to select the threshold that maximised the F1-score. This selected threshold was then applied once to the test set.

Final performance was evaluated on the held-out test set containing 42,894 transactions, including 246 fraud samples. The reported metrics were Accuracy, Precision, Recall, F1-score, ROC-AUC, AUPRC, confusion matrix, and classification report. Since the dataset is highly imbalanced, Precision, Recall, F1-score, and AUPRC were considered more important than Accuracy.

6. Experimental Setup

Component	Details
Platform	Google Colab (GPU runtime)
Python	3.10
PyTorch	2.10.0+cu128
Key Libraries	NumPy, Pandas, scikit-learn, Matplotlib, Seaborn
Hardware	NVIDIA Tesla T4 GPU (CUDA)
Random seed	42 (NumPy, PyTorch, CUDA; cuDNN deterministic)
Optimizer	Adam with $(\beta_1, \beta_2) = (0.5, 0.999)$ for all networks
Epochs	100 for both BiGAN and AnoGAN
BiGAN training time	9.07 min
AnoGAN training time	12.73 min
AnoGAN inference time	Val: 4.47 min, Test: 6.64 min (z-optimisation)

7. Results and Discussion

7.1 Training Convergence

BiGAN training was stable throughout, as shown in Figure 2. The discriminator loss $\mathcal{L}_D$ converged around 1.27, the Generator+Encoder loss settled near 2.95, and the reconstruction loss decreased steadily from 0.49 to 0.23 over 100 epochs, confirming that the Encoder learned to invert the Generator.

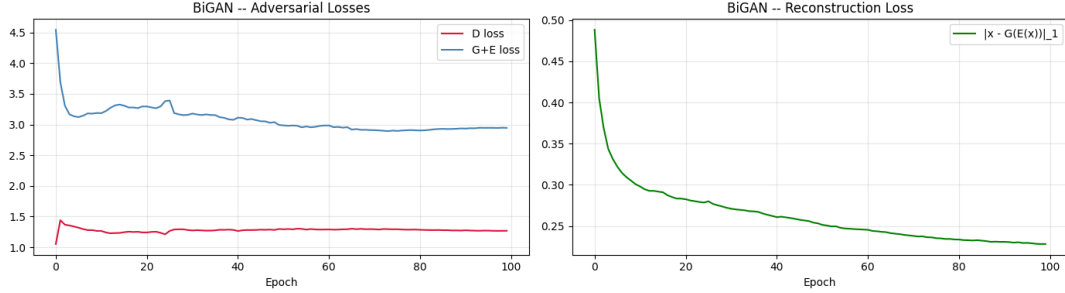


Figure 2: BiGAN training loss curves: adversarial losses (left) and reconstruction loss (right).

AnoGAN training showed the discriminator loss decreasing from 0.64 to 0.24 and the generator loss increasing from 0.88 to 3.94 (Figure 3), consistent with a well-training GAN where the Generator progressively fools a stronger Discriminator.

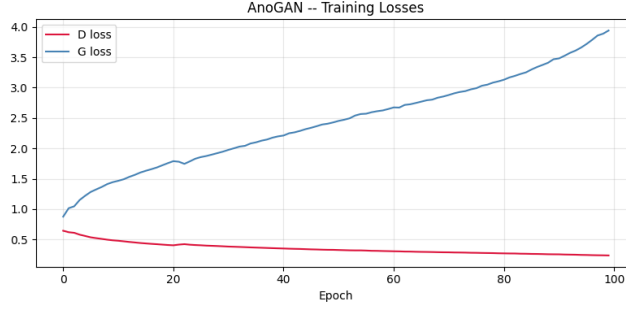


Figure 3: AnoGAN training losses (discriminator and generator over 100 epochs).

7.2 Anomaly Score Distributions

The learned anomaly scores show clear separation between normal and fraud classes on the test set:

	Normal mean $\pm$ std	Fraud mean $\pm$ std	Threshold τ^*
BiGAN	0.183 $\pm$ 0.180	2.481 $\pm$ 1.933	1.191
AnoGAN	1.656 $\pm$ 5.105	149.609 $\pm$ 193.696	21.803

AnoGAN shows a much larger absolute separation (fraud scores $\sim 90\times$ higher than normal), while BiGAN’s separation is more compressed but still discriminative. Both thresholds were selected by maximising F1 on the validation set.

7.3 Per-Model Diagnostic Plots

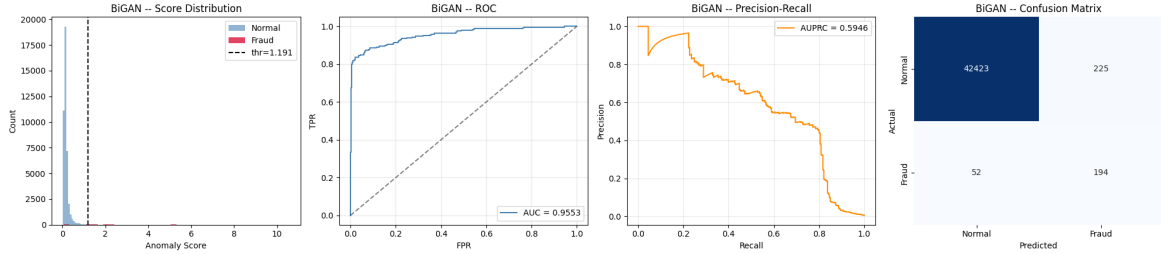


Figure 4: BiGAN test-set diagnostics: score distribution with threshold (left), ROC curve (centre-left), precision-recall curve (centre-right), and confusion matrix (right).

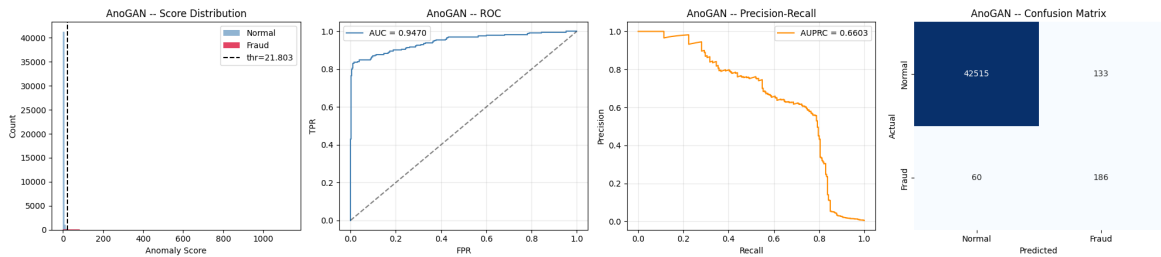


Figure 5: AnoGAN test-set diagnostics: score distribution with threshold (left), ROC curve (centre-left), precision-recall curve (centre-right), and confusion matrix (right).

7.4 Quantitative Results on Test Set

Table 1: Individual model performance on the test set (42,894 samples: 42,648 normal, 246 fraud).

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC	AUPRC
BiGAN	0.9935	0.4630	0.7886	0.5835	0.9553	0.5946
AnoGAN	0.9955	0.5831	0.7561	0.6584	0.9470	0.6603

Confusion matrices:

	BiGAN		AnoGAN	
	Pred Normal	Pred Fraud	Pred Normal	Pred Fraud
Actual Normal	42,423	225	42,515	133
Actual Fraud	52	194	60	186

7.5 BiGAN vs. AnoGAN Comparison

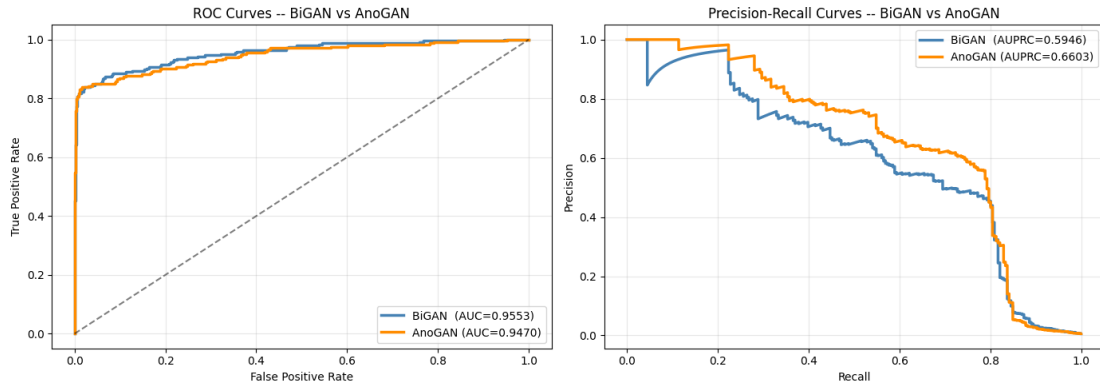


Figure 6: Overlaid ROC curves (left) and precision-recall curves (right) for BiGAN and AnoGAN on the test set.

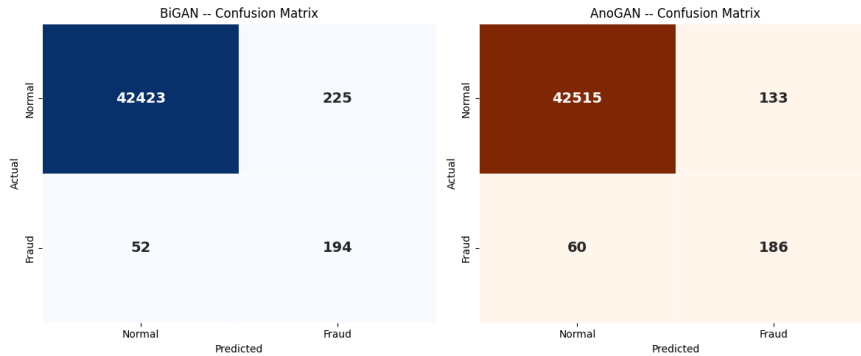


Figure 7: Side-by-side confusion matrices: BiGAN (left, blue) and AnoGAN (right, orange).

Table 2: BiGAN vs. AnoGAN — Summary comparison on the test set.

Metric	BiGAN	AnoGAN	Remarks
Accuracy	0.9935	0.9955	Both high due to class imbalance; not the key metric
Precision	0.4630	0.5831	AnoGAN produces far fewer false alarms
Recall	0.7886	0.7561	BiGAN catches more true frauds
F1-Score	0.5835	0.6584	AnoGAN better balances precision and recall
ROC-AUC	0.9553	0.9470	BiGAN marginally better at ranking fraud
AUPRC	0.5946	0.6603	AnoGAN better at high-precision operating points
Train time	8.90 min	12.13 min	AnoGAN trains slower
Infer. style	Single pass	z-optim. (500 iters)	BiGAN much faster at inference

7.6 Discussion

AnoGAN achieves higher Precision (0.58 vs. 0.46), F1-score (0.66 vs. 0.58), and AUPRC (0.66 vs. 0.59), making it the better overall model when false alarms are costly. Its z-optimisation scoring directly minimises the reconstruction gap, producing a more principled anomaly score with greater separation (fraud scores $\sim 90\times$ normal scores vs. $\sim 14\times$ for BiGAN).

BiGAN achieves slightly higher Recall (0.79 vs. 0.76) and ROC-AUC (0.955 vs. 0.947), meaning it misses fewer frauds at the cost of more false positives. Its key advantage is inference speed: a single encoder forward pass versus 500-iteration z-optimisation per sample in AnoGAN (6.64 min for the test set vs. near-instant).

Both models achieve ROC-AUC > 0.94 and AUPRC far above the random baseline (≈ 0.006), demonstrating that unsupervised GAN-based anomaly detection is viable for credit card fraud even without any fraud labels during training.

8. Algorithmic Summary

Algorithm 1 BiGAN Training and Anomaly Scoring

Input: Normal training data $\mathcal{X}_{\text{train}}$, latent dim d_z , epochs T , λ , w_{lat}

Output: Anomaly scores $\{A(x)\}$ for test samples

- 1: Initialise Generator G , Encoder E , Discriminator D ; Adam optimisers with TTUR
 - 2: **for** epoch = 1 to T **do**
 - 3: **for** each mini-batch $x_{\text{real}} \sim \mathcal{X}_{\text{train}}$ **do**
 - 4: Sample $z_{\text{fake}} \sim \mathcal{N}(0, I)$; compute $\hat{x} = G(z_{\text{fake}})$, $z_{\text{real}} = E(x_{\text{real}})$
 - 5: Add annealed instance noise to x_{real} and $\hat{x}$
 - 6: **D step:** $\mathcal{L}_D = \text{BCE}(D(x_{\text{real}}, z_{\text{real}}), 0.9) + \text{BCE}(D(\hat{x}, z_{\text{fake}}), 0)$
 - 7: **G+E step:** $\mathcal{L}_{GE} = \text{BCE}(D(x_{\text{real}}, z_{\text{real}}), 0) + \text{BCE}(D(\hat{x}, z_{\text{fake}}), 1) + w_r \|x_{\text{real}} - G(E(x_{\text{real}}))\|_1$
 - 8: Clip gradients; update D , then G and E
 - 9: **end for**
 - 10: **end for**
 - 11: **Scoring:** $\hat{z} = E(x)$; $\hat{x} = G(\hat{z})$; $\hat{z}' = E(\hat{x})$
 - 12: $A(x) = (1-\lambda)[\|x - \hat{x}\|_1 + w_{\text{lat}}\|\hat{z} - \hat{z}'\|_1] + \lambda \|f_D(x, \hat{z}) - f_D(\hat{x}, \hat{z})\|_1$
-

Algorithm 2 AnoGAN Training and Anomaly Scoring

Input: Normal training data $\mathcal{X}_{\text{train}}$, latent dim d_z , epochs T , z-itsers K , λ

Output: Anomaly scores $\{A(x^*)\}$ for test samples

- 1: Initialise Generator G , Discriminator D ; Adam optimisers with TTUR
 - 2: **for** epoch = 1 to T **do**
 - 3: **for** each mini-batch $x_{\text{real}} \sim \mathcal{X}_{\text{train}}$ **do**
 - 4: Sample $z \sim \mathcal{N}(0, I)$; $\hat{x} = G(z)$
 - 5: **D step:** $\mathcal{L}_D = 0.5[\text{BCE}(D(x_{\text{real}}), 0.9) + \text{BCE}(D(\hat{x}), 0)]$
 - 6: **G step:** $\mathcal{L}_G = \text{BCE}(D(G(z)), 1)$; clip gradients
 - 7: **end for**
 - 8: **end for**
 - 9: **Scoring (freeze G , D):** Initialise $z \sim \mathcal{N}(0, I)$ with `requires_grad=True`
 - 10: **for** iter = 1 to K **do**
 - 11: $\mathcal{L}(z) = (1-\lambda)\|x^* - G(z)\|_2^2 + \lambda\|f_D(x^*) - f_D(G(z))\|_2^2$; $z \leftarrow z - \nabla_z \mathcal{L}(z)$
 - 12: **end for**
 - 13: $A(x^*) = \mathcal{L}(z^*)$ after convergence
-

9. Conclusion

This assignment implemented two GAN-based unsupervised anomaly detection methods — BiGAN and AnoGAN — for credit card fraud detection on a severely imbalanced dataset (0.17% fraud rate). Both models were trained solely on normal transactions and evaluated using multiple metrics appropriate for imbalanced classification.

- A leak-free two-stage preprocessing pipeline (RobustScaler on **Time/Amount** followed by StandardScaler on all features, fit on training normals only) prevents raw-scale features from dominating the anomaly score.
- **AnoGAN** outperformed BiGAN on Precision (0.583 vs. 0.463), F1-score (0.658 vs. 0.584), and AUPRC (0.660 vs. 0.595), making it the preferred model when minimising false positives is important.
- **BiGAN** achieved higher Recall (0.789 vs. 0.756) and marginally higher ROC-AUC (0.955 vs. 0.947), and is substantially faster at inference due to its single-pass encoder, making it preferable in latency-sensitive deployments.
- Both models achieved ROC-AUC > 0.94 and AUPRC far above the random baseline (≈ 0.006), demonstrating that unsupervised GAN-based anomaly detection is a viable approach for credit card fraud detection even without labelled fraud examples during training.